

Documentación — Módulo 4: Predicción Lineal Simple

Archivo: `modulo_4_prediccion.s`

1. Explicación del algoritmo implementado

El módulo implementa una **predicción lineal simple** sobre los 30 datos históricos de una columna de `lecturas.csv`. Se asume que la tasa de cambio promedio observada entre el primer y el último dato se mantendrá constante, y con eso se estima el siguiente valor de la serie.

El algoritmo sigue tres pasos matemáticos (todo en enteros con signo):

1. Diferencia total entre el primer dato (X_{inicial}) y el último (X_{final}):

$$\text{DIF} = X_{\text{final}} - X_{\text{inicial}}$$

2. Cambio promedio por intervalo, distribuyendo la diferencia total entre los 29 intervalos que separan los 30 datos ($N-1 = 29$):

$$\text{PROMEDIO_CAMBIO} = \text{DIF} / 29$$

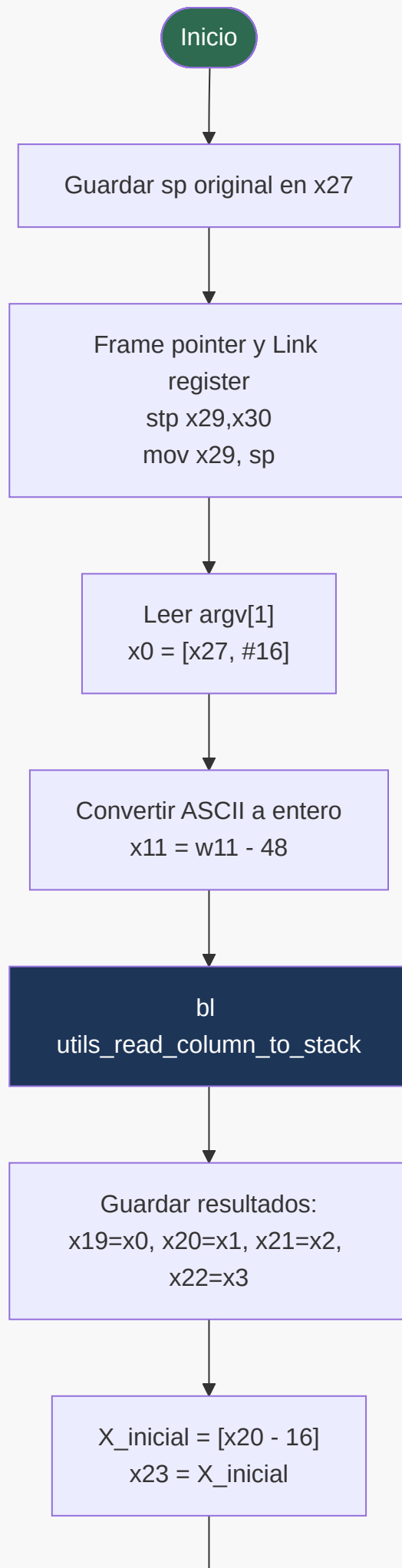
División entera con signo (`sdiv`), el resultado se trunca hacia cero.

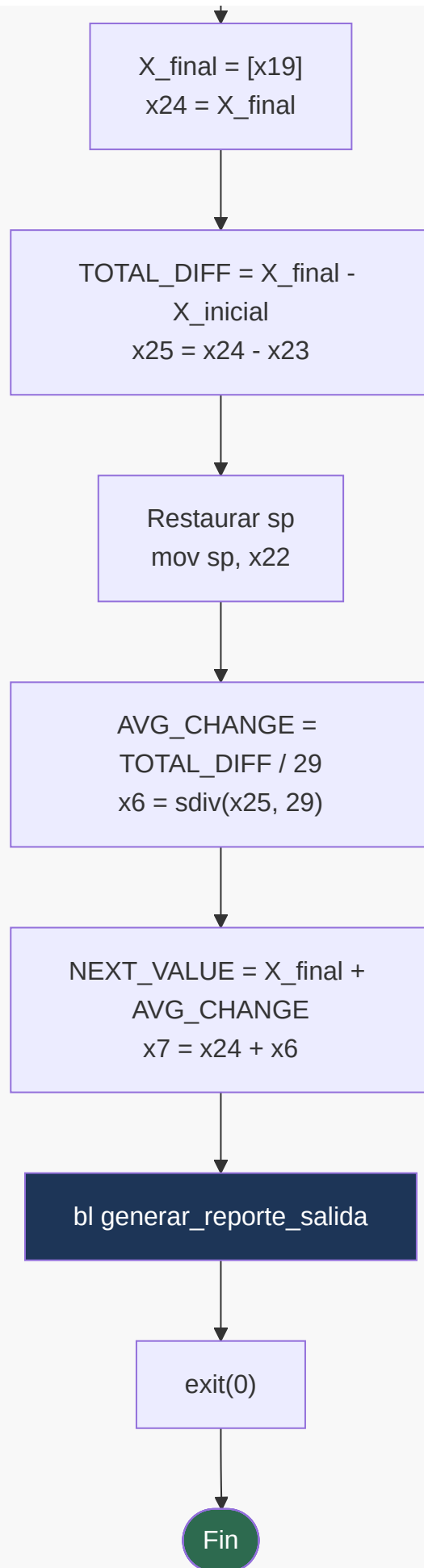
3. Predicción del siguiente valor (dato número 31), asumiendo que la tendencia continúa:

$$\text{PREDICCIÓN} = X_{\text{final}} + \text{PROMEDIO_CAMBIO}$$

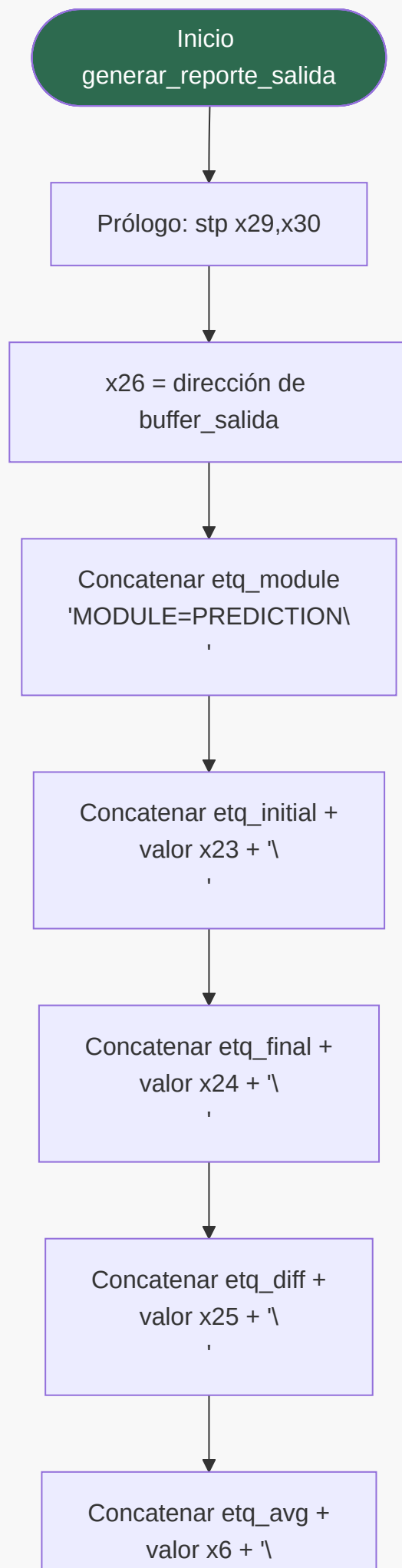
El número de columna a analizar se recibe como argumento (`argv[1]`). El módulo delega en `utils.s` la lectura del CSV y la extracción de los 30 valores de esa columna, realiza los cálculos y escribe el resultado en `resultado_prediccion.txt`.

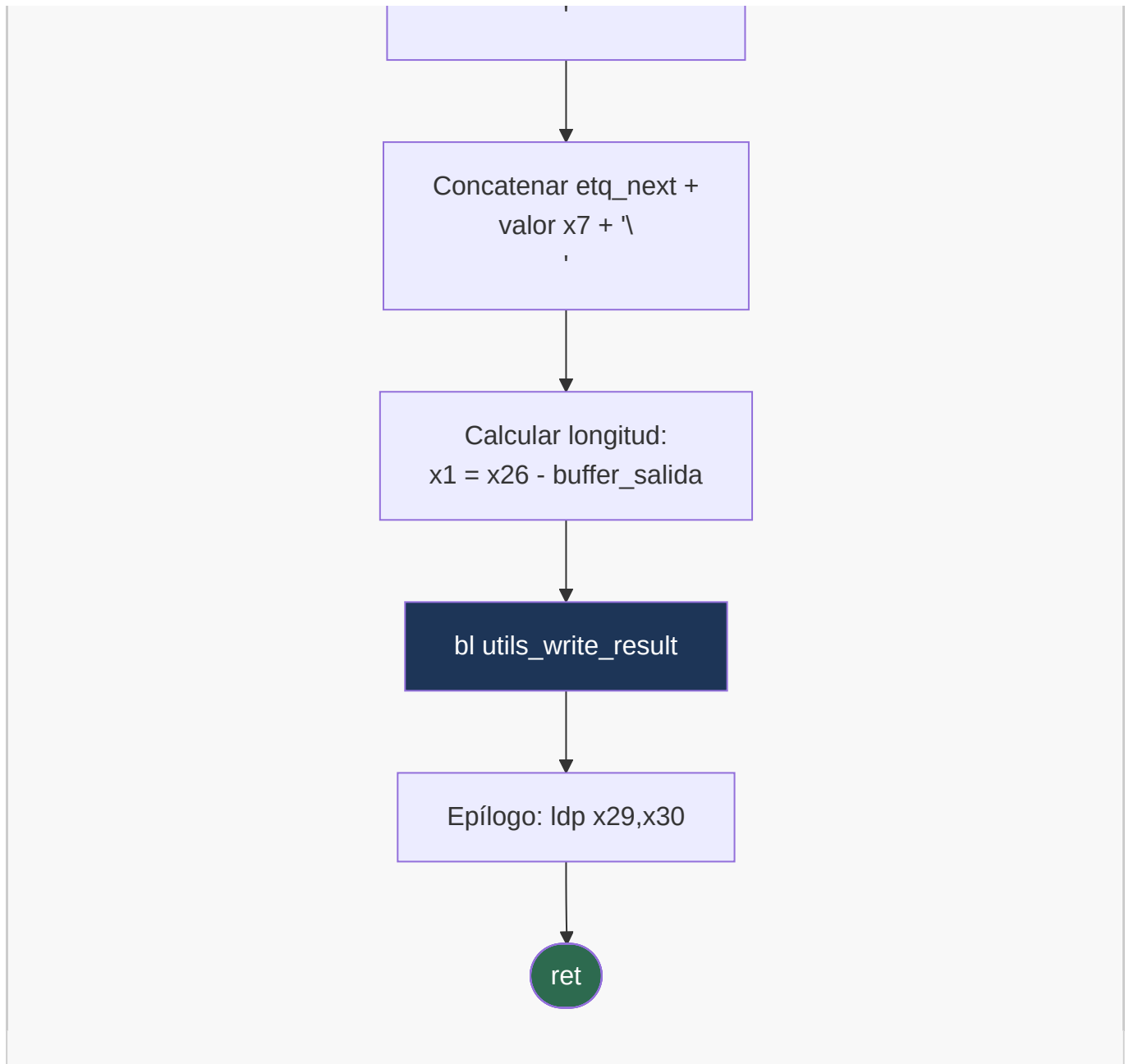
2. Diagrama de flujo



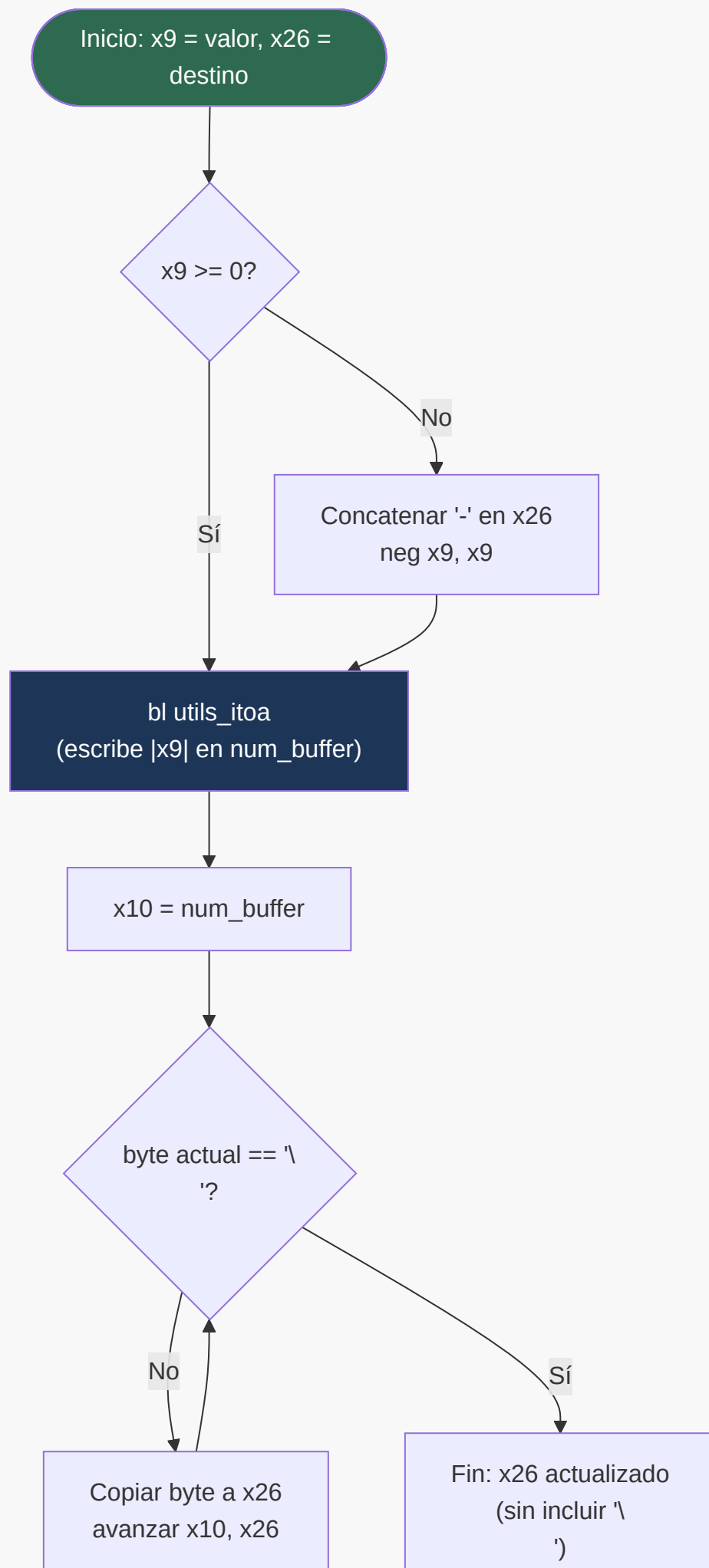


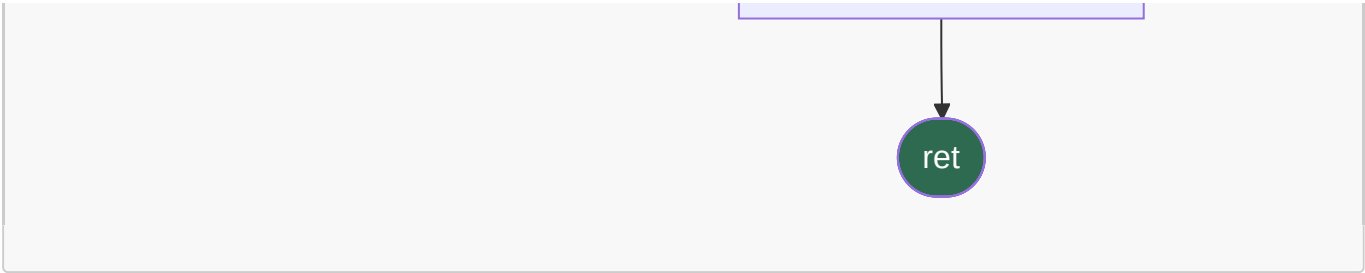
Subrutina `generar_reporte_salida`



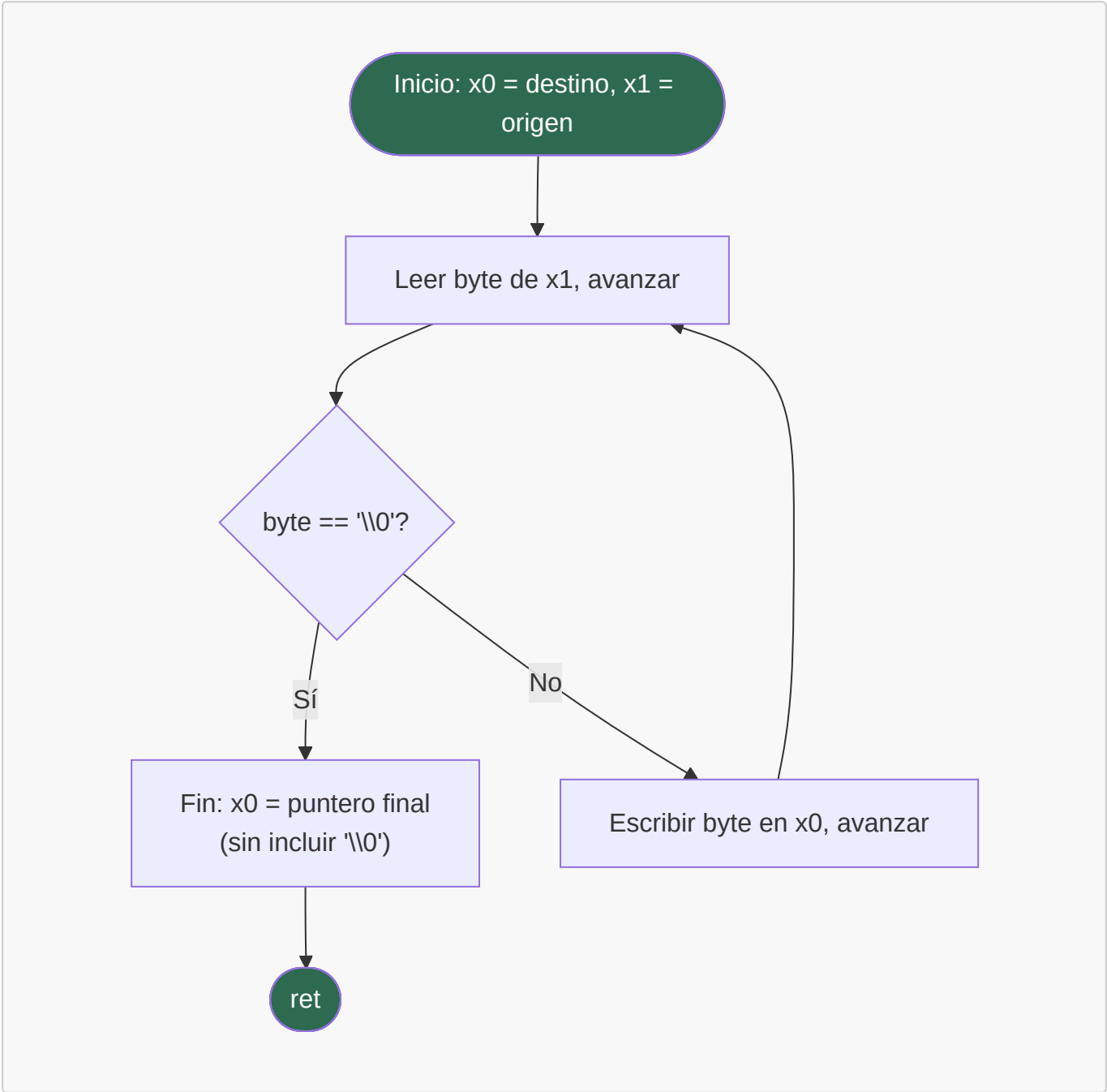


Subrutina `utilidad_escribir_entero`





Subrutina `utilidad_concatenar_cadena`



3. Explicación de los registros utilizados

Registro	Propósito
x27	Copia del <code>sp</code> original al iniciar el programa, antes del prólogo (<code>stp</code>). Apunta al bloque <code>argc/argv</code> que el kernel coloca en el stack al lanzar el proceso.

Registro	Propósito
x29, x30	Frame pointer y link register, guardados en el prólogo (<code>stp x29, x30, [sp, #-16]!</code>) para poder hacer llamadas (<code>bl</code>) sin perder el contexto de retorno.
x11	Número de columna a leer (parámetro de entrada para <code>utils_read_column_to_stack</code>), obtenido de <code>argv[1]</code> .
x19	Dirección más baja del área de datos en el stack devuelta por <code>utils.s</code> (<code>x0</code>). Por el orden en que <code>utils_save_number</code> apila los valores, aquí queda el último dato leído del CSV → <code>X_final</code> .
x20	Dirección más alta del área de datos (<code>x1/x28</code> de <code>utils.s</code>). <code>x20 - 16</code> apunta al primer dato leído del CSV → <code>X_inicial</code> .
x21	Cantidad de datos extraídos (debe ser 30).
x22	Valor de <code>sp</code> original a restaurar al finalizar (<code>x3</code> de <code>utils.s</code>), para dejar la pila balanceada.
x23	<code>X_inicial</code> (primer dato del CSV).
x24	<code>X_final</code> (último dato del CSV).
x25	$DIF = X_final - X_inicial$.
x6	$PROMEDIO_CAMBIO = DIF / 29$.
x7	$PREDICCIÓN = X_final + PROMEDIO_CAMBIO$.
x26	Puntero dinámico que recorre <code>buffer_salida</code> mientras se construye el reporte de texto; se actualiza después de cada concatenación.
x9	Valor entero (con signo) que se pasa a <code>utilidad_escribir_entero</code> para convertir y escribir.
x0-x2, x10, x12	Registros de propósito general usados dentro de las subrutinas auxiliares para punteros, contadores y caracteres temporales (no preservados entre llamadas, de "uso libre" según convención).

4. Explicación de ciclos, saltos y subrutinas

Lectura del argumento (sin ciclo)

```

mov x27, sp
stp x29, x30, [sp, #-16]!
mov x29, sp
ldr x0, [x27, #16]      //x0 = argv[1]
ldrb w11, [x0]          //primer carácter ASCII de argv[1]
sub x11, x11, #48        //ASCII -> entero

```

`x27` se guarda **antes** del `stp` porque éste modifica `sp`. El kernel coloca en el stack, al iniciar el proceso, el bloque `[argc][argv[0]][argv[1]]...`; como `argc` y `argv[0]` ocupan 16 bytes, `argv[1]` está en

sp+16. Se lee solo el primer carácter del argumento (se asume un dígito 1-9) y se convierte de ASCII a entero restando 48 ('0' = 48).

bl utils_read_column_to_stack (subrutina externa)

Llamada única que delega en **utils.s** toda la apertura/lectura del CSV y el parsing de la columna solicitada, devolviendo en **x0-x3** los punteros al área de datos en el stack.

Cálculo de X_inicial y X_final (sin ciclo)

```
sub x0, x20, #16    //dirección de X_inicial
ldr x23, [x0]
ldr x24, [x19]      //X_final
```

No hay ciclo porque solo se necesitan los extremos de la serie (primero y último), no todos los 30 valores intermedios.

generar_reporte_salida (subrutina, secuencia lineal sin ciclos propios)

Construye el reporte concatenando, en orden, 6 bloques de texto: la etiqueta del módulo y 5 pares **ETIQUETA=valor\n**. Cada bloque sigue el mismo patrón:

```
mov x0, x26          //destino = posición actual del buffer
ldr x1, =etiqueta
bl utilidad_concatenar_cadena //copia la etiqueta, devuelve nueva
posición
mov x26, x0
mov x9, <registro_con_valor>
bl utilidad_escribir_entero  //escribe el número, actualiza x26
mov x0, x26
ldr x1, =str_nl
bl utilidad_concatenar_cadena //agrega '\n'
mov x26, x0
```

Al final calcula la longitud total (**x26: dirección_inicial_del_buffer**) y llama a **utils_write_result** para escribir el archivo.

utilidad_escribir_entero (subrutina con ciclo de copia)

Maneja el signo manualmente porque **utils_itoa** solo soporta enteros positivos:

```
cmp x9, #0
bge .ue_positivo    //salto condicional: si x9 >= 0, omitir el signo
... escribir '-' y neg x9, x9 ...
.ue_positivo:
    bl utils_itoa      //convierte |x9| a "digitos\n\0" en num_buffer
    ldr x10, =num_buffer
```

```
.ue_copy:                //ciclo: copia carácter por carácter
    ldrb w12, [x10], #1
    cmp w12, #10          //¿es '\n'?
    beq .ue_fin           //salto condicional: termina sin copiar el '\n'
    strb w12, [x26], #1
    b .ue_copy            //salto incondicional: repetir
.ue_fin:
```

El ciclo `.ue_copy` recorre el resultado de `utils_itoa` y lo transfiere al buffer de salida, **deteniéndose antes del `\n`** que `utils_itoa` siempre agrega, porque el formato `CLAVE=VALOR\n` ya añade su propio salto de línea por separado.

`utilidad_concatenar_cadena` (subrutina con ciclo de copia)

```
.loop_copia_bytes:
    ldrb w2, [x1], #1      //lee byte de origen, avanza
    cbz w2, .fin_copia_bytes //salto condicional: si es '\0', termina
    strb w2, [x0], #1      //escribe byte en destino, avanza
    b .loop_copia_bytes    //salto incondicional: repetir
.fin_copia_bytes:
    ret
```

Ciclo clásico de copia de string terminado en `\0`, usado para transferir las etiquetas estáticas (`etq_module`, `etq_initial`, etc.) y los caracteres `\n` / `-` al buffer dinámico de salida.

5. Explicación del formato de entrada y salida

Entrada

- **Archivo:** `lecturas.csv` (leído internamente por `utils_read_column_to_stack`, no por este módulo directamente).
- **Formato del CSV:** encabezado seguido de 30 filas de 9 columnas separadas por comas, terminado en un carácter `$`:

```
ID, TEMP, HUM_AIRE, HUM_SUELO_1, HUM_SUELO_2, LUZ, GAS, RIEGO_1, RIEGO_2
1, 31, 65, 60, 18, 527, 50, 0, 0
...
30, 27, 74, 58, 15, 347, 91, 0, 0
$
```

Salida

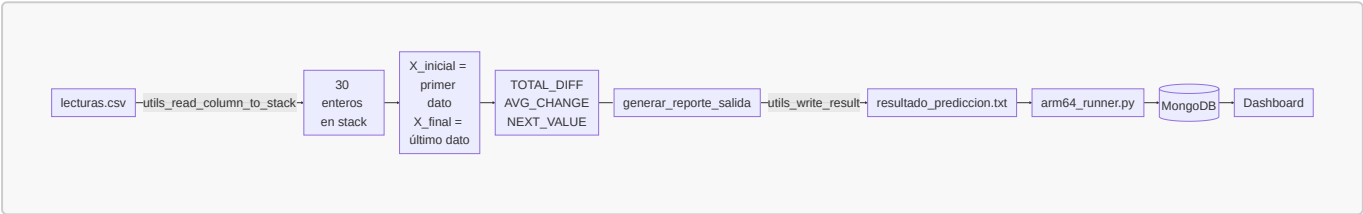
- **Archivo:** `resultado_prediccion.txt`, texto plano en formato `CLAVE=VALOR`, una por línea, leído posteriormente por el backend para almacenarse en MongoDB y mostrarse en el dashboard.

- Ejemplo de salida real (columna LUZ):

```
MODULE=PREDICTION
INITIAL_VALUE=527
FINAL_VALUE=347
TOTAL_DIFF=-180
AVG_CHANGE=-6
NEXT_VALUE=341
```

- Todos los valores son **enteros con signo**; los valores negativos se representan con un - antepuesto.

6. Flujo general resumido



7. Nota sobre manejo de memoria

El módulo no reserva memoria propia para los 30 datos: reutiliza directamente el área del stack que `utils_read_column_to_stack` deja apilada (`x19/x20`), y restaura `sp` (`mov sp, x22`) en cuanto extrae `X_inicial` y `X_final`, dejando la pila balanceada antes de construir el reporte.

La única memoria propia declarada está en `.bss`:

Buffer	Tamaño	Uso
<code>num_buffer</code>	32 bytes	Conversión temporal de enteros a texto (<code>utils_itoa</code>)
<code>buffer_salida</code>	512 bytes	Reporte completo antes de escribirlo a disco

8. Depuración GDB

Declaración de primeros breakpoints recorrido de algunas líneas de código

```
difer@difer-IdeaPad-Slim-3-15IAH8:~/Documentos/Diego Usac (linux)/2026/Vacaciones_Junio/A'
rqui/Lab/Proyecto 1/Pruebas personales$ gdb-multiarch ./modulo_4_prediccion
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./modulo_4_prediccion...
(gdb) break _start
Punto de interrupción 1 at 0x4000b0: file modulo_4_prediccion.s, line 40.
(gdb) target remote :1234
Remote debugging using :1234

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
_start () at modulo_4_prediccion.s:40
40      mov x27, sp                //x27 = sp original (apunta a argc/argv)
(gdb) break generar_reporte_salida
Punto de interrupción 2 at 0x40010c: file modulo_4_prediccion.s, line 84.
(gdb) stepi
41      stp x29, x30, [sp, #-16]!
(gdb) stepi
_start () at modulo_4_prediccion.s:42
42      mov x29, sp
(gdb) stepi
44      ldr x0, [x27, #16]         //x0 = argv[1]
(gdb) print #x0
✗ Invalid character '#' in expression.
(gdb) print $x0
$1 = 0
(gdb) stepi
45      ldrb w11, [x0]            //primer carácter ASCII
(gdb) print $x0
$2 = 134793186504261
(gdb) stepi
46      sub x11, x11, #48         //ASCII -> entero -> x11 = número de columna
(gdb) stepi
49      bl utils_read_column_to_stack
(gdb) print $x11
$3 = 6
(gdb) stepi
utils_read_column_to_stack () at utils.s:51
```

Se recorren más líneas de código para comprobar que todo está en orden

```
difer@difer-IdeaPad-Slim-3-15IAH8:~/Documentos/Diego Usac (linux)/2026/Vacaciones_Junio/A'
rqui/Lab/Proyecto 1/Pruebas personales$ gdb-multiarch ./modulo_4_prediccion
$3 = 6
(gdb) stepi
utils_read_column_to_stack () at utils.s:51
51      stp x29, x30, [sp, #-16]! // Guardar frame pointer (x29) y link register (x
30)
(gdb) stepi
52      mov x29, sp                // Actualizar frame pointer al stack actual
(gdb) stepi
54      mov x28, sp                // x28 = límite superior de datos (dirección más
alta)
(gdb) stepi
55      add x27, x28, #16          // x27 = posición segura para restaurar el stack
original
(gdb) stepi
57      mov x22, #0                // x22 = contador de números extraídos (llegará a
30)
(gdb) stepi
59      bl utils_open_file         // Llamada para abrir lecturas.csv
(gdb) stepi
utils_open_file () at utils.s:151
151     mov x0, #-100              // AT_FDCWD (indica que busque en el directorio a
ctual)
(gdb) stepi
152     ldr x1, =filename          // Cargar dirección del nombre del archivo "lectu
ras.csv"
(gdb) stepi
153     mov x2, #0                 // Bandera O_RDONLY (0 = solo lectura)
(gdb) stepi
154     mov x3, #0                 // Modo (sin permisos extra al solo leer)
(gdb) stepi
155     mov x8, #56                 // Syscall 56: openat
(gdb) stepi
156     svc #0                     // Interrupción de software
(gdb) stepi
159     blt utils_open_error       // Si es negativo, hubo error, saltar a rutina de
fallo
(gdb) stepi
161     mov x19, x0                // Guardar exitosamente el File Descriptor en x19
(gdb) stepi
162     ret                        // Retornar al hilo principal
```

Se comprueba que los valores extraídos del .csv y que los resultantes de la operación de predicción estén correctamente guardados y sean acertados

```
difer@difer-IdeaPad-Slim-3-15IAH8:~/Documentos/Diego Usac (linux)/2026/Vacaciones_Junio/A
rqui/Lab/Proyecto 1/Pruebas personales$ gdb-multiarch ./modulo_4_prediccion
162      ret                                // Retornar al hilo principal
(gdb) stepi
utils_read_column_to_stack () at utils.s:60
60      bl utils_read_file                // Llamada para volcar el contenido al buffer
(gdb) stepi
utils_read_file () at utils.s:165
165      mov x0, x19                      // Param: File Descriptor del archivo abierto
(gdb) stepi
166      ldr x1, =buffer                  // Param: Dirección de memoria destino (bss buffe
r)
(gdb) stepi
167      mov x2, #4096                    // Param: Tamaño máximo a leer en bytes
(gdb) stepi
168      mov x8, #63                      // Syscall 63: read
(gdb) stepi
169      svc #0                          // Ejecutar syscall
(gdb) continue
Continuando.

Breakpoint 2, generar_reporte_salida () at modulo_4_prediccion.s:84
84      stp x29, x30, [sp, #-16]!
(gdb) print $x19
$4 = 134793186502816
(gdb) print $x21
$5 = 30
(gdb) print $x23
$6 = 527
(gdb) print $x24
$7 = 347
(gdb) print $x25
$8 = -180
(gdb) print $x6
$9 = -6
(gdb) print $x7
$10 = 341
(gdb) break utilidad_escribir_entero
Punto de interrupción 3 at 0x40020c: file modulo_4_prediccion.s, line 168.
(gdb) stepi
85      mov x29, sp
(gdb) break utilidad_escribir_entero
```

Como todo pareciera estar en orden, se finaliza la depuración

```
difer@difer-IdeaPad-Slim-3-15IAH8:~/Documentos/Diego Usac (linux)/2026/Vacaciones_Junio/A
rqui/Lab/Proyecto 1/Pruebas personales$ gdb-multiarch ./modulo_4_prediccion
(gdb) break utilidad_escribir_entero
Nota: punto de interrupción 3 also set at pc 0x40020c.
Punto de interrupción 4 at 0x40020c: file modulo_4_prediccion.s, line 168.
(gdb) stepi
87      ldr x26, =buffer_salida
(gdb) stepi
90      mov x0, x26
(gdb) print $x26
$11 = 4261456
(gdb) stepi
91      ldr x1, =etq_module
(gdb) print $x0
$12 = 4261456
(gdb) stepi
92      bl  utilidad_concatenar_cadena
(gdb) stepi
utilidad_concatenar_cadena () at modulo_4_prediccion.s:199
199      ldrb w2, [x1], #1
(gdb) stepi
200      cbz w2, .fin_copia_bytes
(gdb) stepi
201      strb w2, [x0], #1
(gdb) stepi
202      b   .loop_copia_bytes
(gdb) stepi
199      ldrb w2, [x1], #1
(gdb) stepi
200      cbz w2, .fin_copia_bytes
(gdb) continue
Continuando.

Breakpoint 3, utilidad_escribir_entero () at modulo_4_prediccion.s:168
168      stp x29, x30, [sp, #-16]!
(gdb) continue
Continuando.

Breakpoint 3, utilidad_escribir_entero () at modulo_4_prediccion.s:168
168      stp x29, x30, [sp, #-16]!
(gdb) continue
Continuando.
```